

Проектирование информационной архитектуры и
взаимодействия с пользователем

Лабораторная работа №4

Лушакова Светлана, Мишкова Диана, Шибко Татьяна
4 курс 12 группа

Задание 1. Проектирование модели требований и реестра вариантов использования

1. Изучить статью «[Проектирование REST API на примере интернет-магазина](#)», материал которой используется в качестве основы для реализации последующих заданий.
2. На основе требований из лабораторной 3 описать требования в виде модели (см. рис. 1 ниже): Действующее лицо → User Story → Use Case

Актор	User Story	Use Case
Покупатель	US-1.1 Как покупатель интернет-магазина, я хочу видеть перечень товаров, чтобы узнать подробности ассортимента (название товара, поставщик, стоимость, количество единиц в наличии)	UC 1.1 Посмотреть список товаров
Покупатель	US-1.2 Как покупатель интернет-магазина, я хочу видеть перечень поставщиков, чтобы узнать подробности о продавцах (название компании, телефон, емейл, адрес)	UC 1.2 Посмотреть список поставщиков
Менеджер	US-2.1.1 Как менеджер интернет-магазина, я хочу видеть перечень товаров с их данными (название, поставщик, стоимость, количество единиц в наличии), чтобы повысить эффективность товарного ассортимента	UC 1.1 Посмотреть список товаров

Рисунок 1

Актор	User Story	Use Case
Клиент	US-1.1 Как клиент, я хочу просматривать меню ресторана, чтобы выбрать блюда.	UC 1.1 Посмотреть меню
Клиент	US-1.2 Как клиент, я хочу просматривать отзывы о еде и обслуживании, чтобы	UC 1.2 Посмотреть отзывы

	иметь представление о заведении.	
Клиент	US-1.3 Как клиент, я хочу оставлять отзывы о еде и обслуживании, чтобы поделиться своим опытом.	UC 1.3 Оставить отзыв
Клиент	US-1.4 Как клиент, я хочу знать об акциях в заведениях, чтобы заказывать выгодно.	UC 1.4 Посмотреть акции
Клиент	US-1.5 Как клиент, я хочу просматривать свою историю заказов, чтобы при необходимости повторить.	UC 1.5 Посмотреть историю заказов
Клиент	US-1.6 Как клиент, я хочу добавлять блюда в корзину, чтобы сделать заказ.	UC 1.6 Добавить блюдо в корзину UC 1.7 Сделать заказ
Клиент	US-1.7 Как клиент, я хочу забронировать место в ресторане.	UC 1.8 Забронировать место
Администратор	US-2.1 Как администратор, я хочу просматривать меню ресторана.	UC 1.1 Посмотреть меню
Администратор	US-2.2 Как администратор ресторана, я хочу управлять меню (добавлять, изменять, удалять блюда), чтобы поддерживать актуальность предложения.	UC 2.1 Добавить блюдо UC 2.2 Изменить блюдо UC 2.3 Удалить блюдо

Администратор	US-2.3 Как администратор, я хочу просматривать отзывы о еде и обслуживании.	UC 1.2 Посмотреть отзывы
Администратор	US-2.4 Как администратор, я хочу просматривать список заказов, чтобы оценивать эффективность заведения по состояниям заказов.	UC 2.4 Посмотреть список заказов
Администратор	US-2.5 Как администратор, я хочу изменять состояние клиентского заказа, чтобы информировать клиента о его готовности.	UC 2.5 Посмотреть заказ UC 2.6 Изменить состояние заказа
Администратор	US-2.6 Как администратор, я хочу подтверждать бронь, чтобы контролировать поток клиентов.	UC 2.4 Подтвердить бронь

3. Добавить модель требований и реестр вариантов использования в документацию проекта в вики и github pages.

Requirements Model and Use Case Registry

Requirements Model

Актор	User Story	Use Case
Клиент	US-1.1 Как клиент, я хочу просматривать меню ресторана, чтобы выбрать блюда.	UC 1.1 Посмотреть меню
Клиент	US-1.2 Как клиент, я хочу просматривать отзывы о еде и обслуживании, чтобы иметь представление о заведении.	UC 1.2 Посмотреть отзывы
Клиент	US-1.3 Как клиент, я хочу оставлять отзывы о еде и обслуживании, чтобы поделиться своим опытом.	UC 1.3 Оставить отзыв
Клиент	US-1.4 Как клиент, я хочу знать об акциях в заведениях, чтобы заказывать выгодно.	UC 1.4 Посмотреть акции
Клиент	US-1.5 Как клиент, я хочу просматривать свою историю заказов, чтобы при необходимости повторить.	UC 1.5 Посмотреть историю заказов
Клиент	US-1.6 Как клиент, я хочу добавлять блюда в корзину, чтобы сделать заказ.	UC 1.6 Добавить блюдо в корзину, UC 1.7 Сделать заказ
Клиент	US-1.7 Как клиент, я хочу забронировать место в ресторане.	UC 1.8 Забронировать место
Администратор	US-2.1 Как администратор, я хочу просматривать меню ресторана.	UC 1.1 Посмотреть меню
Администратор	US-2.2 Как администратор ресторана, я хочу управлять меню (добавлять, изменять, удалять блюда), чтобы поддерживать актуальность предложения.	UC 2.1 Добавить блюдо, UC 2.2 Изменить блюдо, UC 2.3 Удалить блюдо
Администратор	US-2.3 Как администратор, я хочу просматривать отзывы о еде и обслуживании.	UC 1.2 Посмотреть отзывы
Администратор	US-2.4 Как администратор, я хочу просматривать список заказов, чтобы оценивать эффективность заведения по состояниям заказов.	UC 2.4 Посмотреть список заказов
	US-2.5 Как администратор, я хочу изменять состояние	UC 2.5 Посмотреть заказ, UC 2.6

Specification · Asian-Restaurant/general Wiki		
Администратор	список заказов, чтобы оценивать эффективность заведения по состояниям заказов.	UC 2.4 Посмотреть список заказов
Администратор	US-2.5 Как администратор, я хочу изменять состояние клиентского заказа, чтобы информировать клиента о его готовности.	UC 2.5 Посмотреть заказ, UC 2.6 Изменить состояние заказа
Администратор	US-2.6 Как администратор, я хочу подтверждать бронь, чтобы контролировать поток клиентов.	UC 2.7 Подтвердить бронь

Use Case Registry

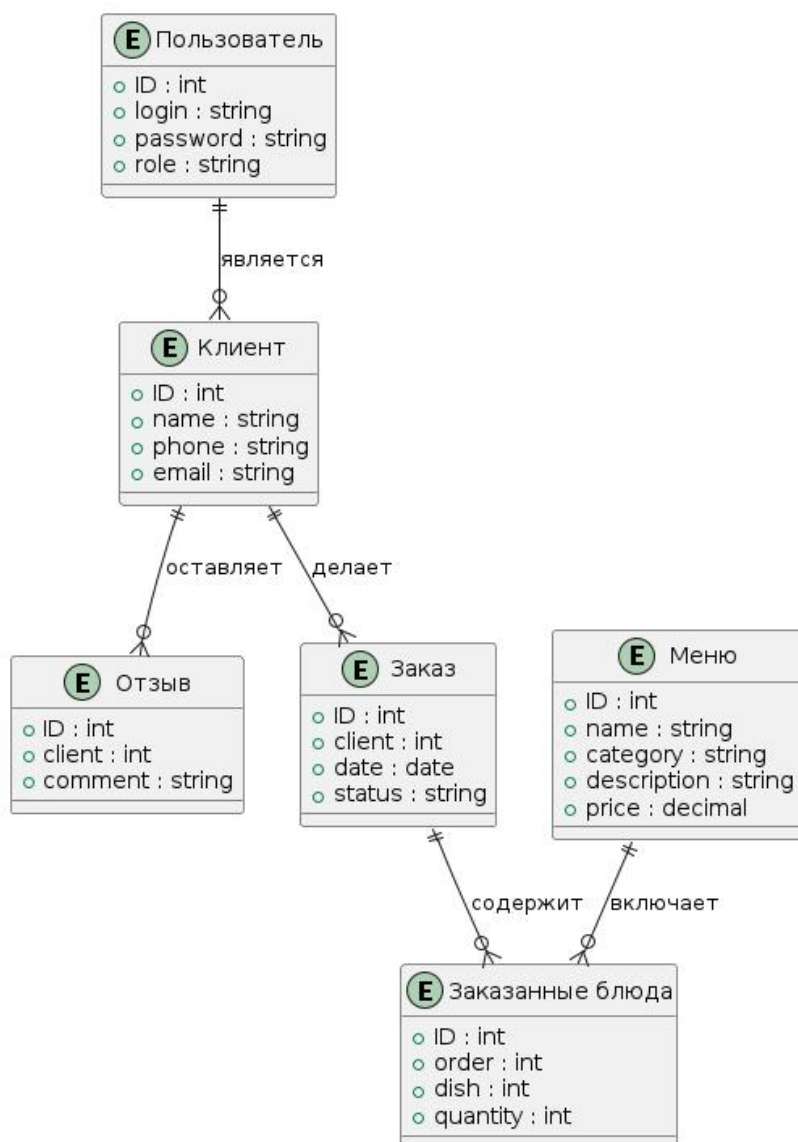
Актор	Use Case
Пользователь	UC 0 Войти в систему
Пользователь	UC 0.1 Аутентификация
Пользователь	UC 0.2 Зарегистрироваться
Пользователь	UC 1.1 Посмотреть меню
Пользователь	UC 1.2 Посмотреть отзывы
Клиент	UC 1.3 Оставить отзыв
Клиент	UC 1.4 Посмотреть акции
Клиент	UC 1.5 Посмотреть историю заказов
Клиент	UC 1.6 Добавить блюдо в корзину
Клиент	UC 1.7 Сделать заказ
Клиент	UC 1.8 Забронировать место
Администратор	UC 2.1 Добавить блюдо
Администратор	UC 2.2 Изменить блюдо
Администратор	UC 2.3 Удалить блюдо
Администратор	UC 2.4 Посмотреть список заказов
Администратор	UC 2.5 Посмотреть заказ
Администратор	UC 2.6 Изменить состояние заказа
Администратор	UC 2.7 Подтвердить бронь

Задание 2. Проектирование физической модели базы данных

1. Уточнить логическую модель базы данных, созданную в 3-ей лаб. работе в виде ER-диаграммы, и представить в виде логической схемы вида (см. рис. 2):

Сущность	Таблица БД	Поле	Смысл поля	Тип данных
Товар	product	id	уникальный идентификатор товара	INTEGER
		name	название товара	VARCHAR(255)
		provider	уникальный идентификатор поставщика	INTEGER
		price	цена товара	
		quantity	количество товара на складе	INTEGER
Поставщик	provider	id	уникальный идентификатор поставщика	INTEGER
		name	название поставщика	VARCHAR(255)
		phone	контактный номер телефона поставщика	VARCHAR(255)
		email	электронная почта поставщика	VARCHAR(255)
		address	адрес поставщика	VARCHAR(255)

Рисунок 2



Сущность	Таблица БД	Поле	Смысл поля	Тип данных
Пользователь	users	ID	Уникальный идентификатор пользователя	INTEGER
		login	Логин пользователя	VARCHAR(255)
		password	Пароль пользователя	VARCHAR(255)
		role	Роль пользователя	VARCHAR(255)
Клиент	clients	ID	Уникальный идентификатор клиента	INTEGER
		name	Имя клиента	VARCHAR(255)
		phone	Телефонный номер клиента	VARCHAR(255)
		email	Электронная почта клиента	VARCHAR(255)
		user_id	Ссылка на пользователя	INTEGER
Меню	menu	ID	Уникальный идентификатор блюда	INTEGER
		name	Название блюда	VARCHAR(255)
		category	Категория блюда	VARCHAR(255)

		description	Описание блюда	TEXT
		price	Цена блюда	DECIMAL
Отзыв	reviews	ID	Уникальный идентификатор отзыва	INTEGER
		client	Клиент-автор отзыва	INTEGER
		comment	Текст отзыва	TEXT
Заказ	orders	ID	Уникальный идентификатор заказа	INTEGER
		client	Клиент-заказчик	INTEGER
		date	Дата оформления заказа	DATE
		status	Статус заказа	VARCHAR(255)
Заказанные блюда	ordered_dishes	ID	Уникальный идентификатор записи заказа	INTEGER
		order	Ссылка на заказ	INTEGER
		dish	Входящее блюдо	INTEGER
		quantity	Количество заказанных блюд	INTEGER

2. Разработать физическую модель данных с учетом используемой СУБД в проекте и представить в виде диаграммы ER как на рисунке 3 и файла в формате .sql.

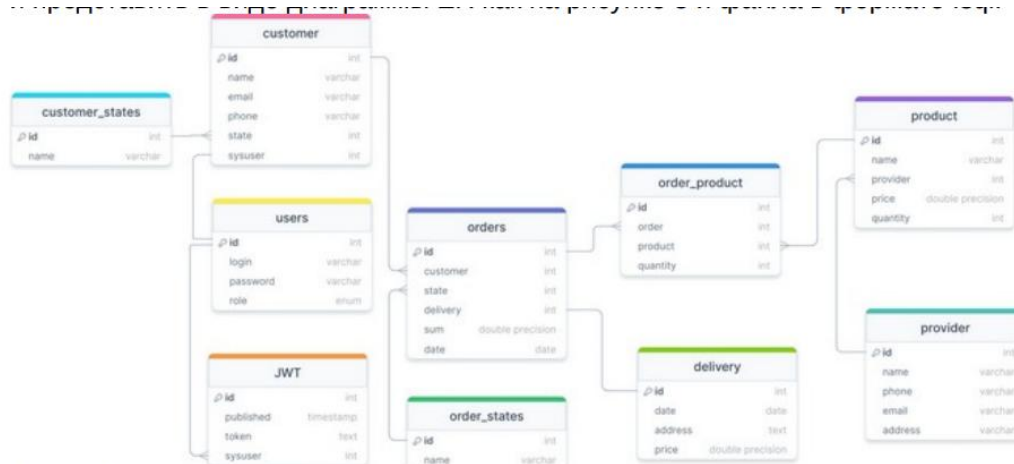


Рисунок 3

```

sviatlana@Sviatlana: ~
mysql> CREATE DATABASE asian_paradise_db;
Query OK, 1 row affected (0,01 sec)

mysql> USE asian_paradise_db;
Database changed
mysql> CREATE TABLE Users (
  ->   ID INT PRIMARY KEY AUTO_INCREMENT,
  ->   login VARCHAR(255) NOT NULL UNIQUE,
  ->   password VARCHAR(255) NOT NULL,
  ->   role VARCHAR(255) NOT NULL
  -> );
Query OK, 0 rows affected (0,05 sec)

mysql>
mysql> CREATE TABLE Clients (
  ->   ID INT PRIMARY KEY AUTO_INCREMENT,
  ->   name VARCHAR(255) NOT NULL,
  ->   phone VARCHAR(255) NOT NULL UNIQUE,
  ->   email VARCHAR(255) NOT NULL UNIQUE
  -> );
Query OK, 0 rows affected (0,02 sec)

mysql>
mysql> CREATE TABLE Menu (
  ->   ID INT PRIMARY KEY AUTO_INCREMENT,
  ->   name VARCHAR(255) NOT NULL,
  ->   category VARCHAR(255) NOT NULL,
  ->   description TEXT,
  ->   price DECIMAL(10, 2) NOT NULL
  -> );
Query OK, 0 rows affected (0,02 sec)

mysql>
mysql> CREATE TABLE Reviews (
  ->   ID INT PRIMARY KEY AUTO_INCREMENT,
  ->   client INT NOT NULL,
  ->   comment TEXT NOT NULL,
  ->   FOREIGN KEY (client) REFERENCES Clients(ID)
  -> );
Query OK, 0 rows affected (0,03 sec)
    
```

```
CREATE DATABASE asian_paradise_db;

USE asian_paradise_db;

CREATE TABLE Users (
    ID INT PRIMARY KEY AUTO_INCREMENT,
    login VARCHAR(255) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    role VARCHAR(255) NOT NULL
);

CREATE TABLE Clients (
    ID INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT UNIQUE,
    name VARCHAR(255) NOT NULL,
    phone VARCHAR(255) NOT NULL UNIQUE,
    email VARCHAR(255) NOT NULL UNIQUE,
    FOREIGN KEY (user_id) REFERENCES Users(ID)
);

CREATE TABLE Menu (
    ID INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(255) NOT NULL,
    category VARCHAR(255) NOT NULL,
    description TEXT,
    price DECIMAL(10, 2) NOT NULL
);

CREATE TABLE Reviews (
    ID INT PRIMARY KEY AUTO_INCREMENT,
    client INT NOT NULL,
    comment TEXT NOT NULL,
    FOREIGN KEY (client) REFERENCES Clients(ID)
```

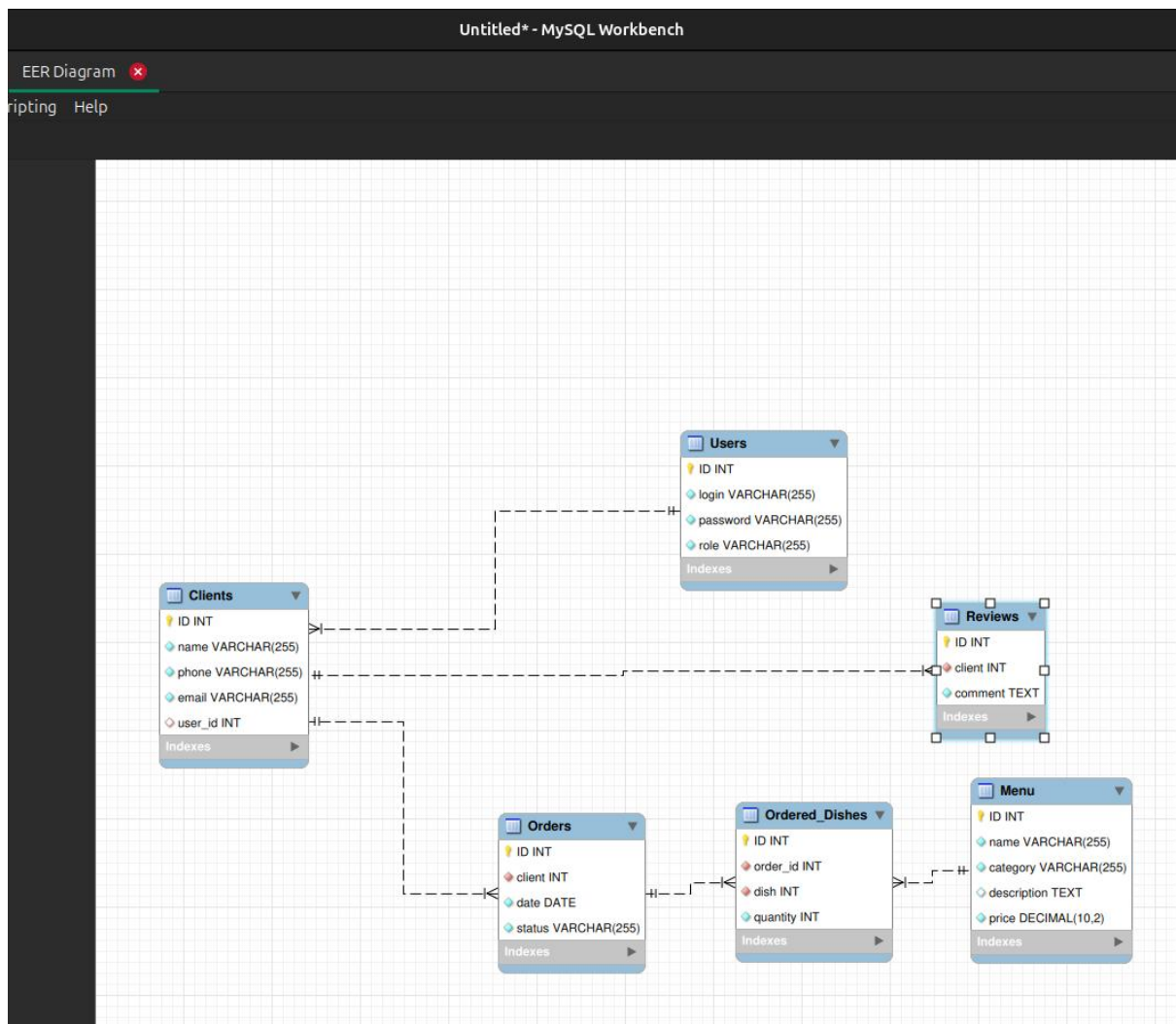
);

```
CREATE TABLE Orders (  
  ID INT PRIMARY KEY AUTO_INCREMENT,  
  client INT NOT NULL,  
  date DATE NOT NULL,  
  status VARCHAR(255) NOT NULL,  
  FOREIGN KEY (client) REFERENCES Clients(ID)  
);
```

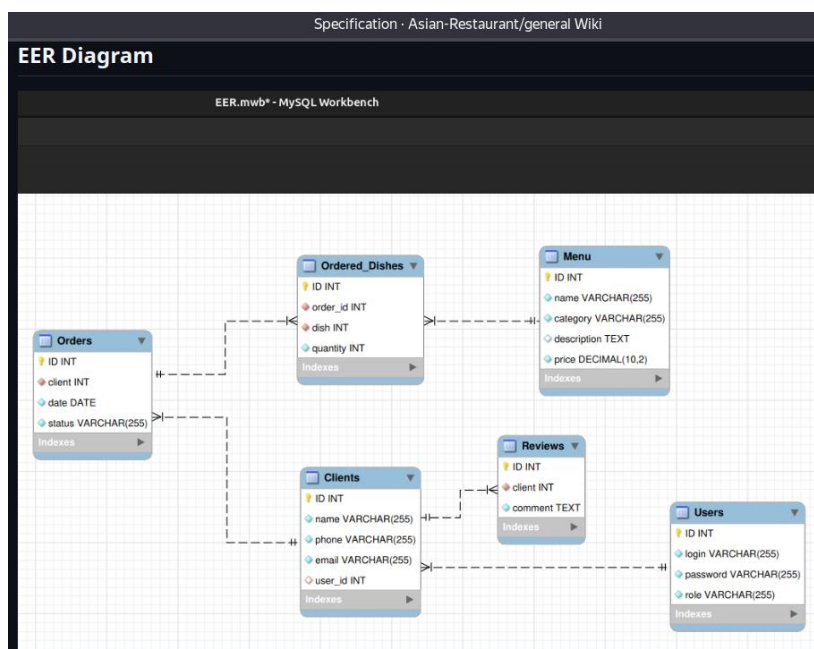
```
CREATE TABLE Ordered_Dishes (  
  ID INT PRIMARY KEY AUTO_INCREMENT,  
  order_id INT NOT NULL,  
  dish INT NOT NULL,  
  quantity INT NOT NULL,  
  FOREIGN KEY (order_id) REFERENCES Orders(ID),  
  FOREIGN KEY (dish) REFERENCES Menu(ID)  
);
```

SHOW TABLES;

```
mysql> SHOW TABLES;  
+-----+  
| Tables_in_asian_paradise_db |  
+-----+  
| Clients                     |  
| Menu                        |  
| Ordered_Dishes              |  
| Orders                      |  
| Reviews                     |  
| Users                       |  
+-----+  
6 rows in set (0,00 sec)  
  
mysql>
```



3. Добавить логическую и физическую модель базы данных в документацию проекта в вики и github pages.



Задание 3. Проектирование маршрутов и конечных точек в виде спецификации API

1. На основе материалов статьи, приведенной в задании 1 выполнить проектирование, маршрутов и конечных точек и представить их в виде как на рисунке 4.

Сущность	Таблица БД	Маршрут
Блюдо	dishes	/menu/dish/{id}
Заказ	orders	/order
Отзыв	feedback	/feedback

2. Сопоставить варианты использования с конечными точками, т.е. HTTP-методами, которые будут обращаться к маршрутам как на рисунке 5:

Актор	Use Case	Маршрут	HTTP-запрос	Аутентификация
Клиент	Просмотр меню	/menu	GET	нет
Клиент	Просмотр акций	/promotions	GET	нет
Клиент	Просмотр категорий меню	menu/categories	GET	нет
Клиент	Просмотр деталей блюда	/menu/dish/{id}	GET	нет
Клиент	Подтверждение заказа	/order/submit	POST	нет
Сотрудник	Обзор всех заказов	/orders	GET	да
Сотрудник	Отправка заказа	/order/dispatch/{id}	POST	да

Сотрудник	Добавить блюдо	/menu/dish/{id}	POST	да
Сотрудник	Изменить состояние заказа	/order/update_status/{id}	PUT	да
Сотрудник	Изменить блюдо	/menu/update_dish/{id}	PUT	да
Сотрудник	Удалить блюдо	/menu/delete_dish/{id}	DELETE	да
Клиент	Написать отзыв	/feedback/review	POST	нет
Клиент	Просмотр отзывов	feedback/reviews	GET	нет

3. Добавить описание маршрутов и конечных точек в документацию проекта в вики и github pages.

Подробное описание маршрутов

Diana Mishkova edited this page now · [1 revision](#)

1. /menu - Просмотр меню

- HTTP Метод: GET
- Аутентификация: Нет
- Описание: Этот маршрут позволяет клиентам просматривать меню ресторана.

2. /promotions - Просмотр акций

- HTTP Метод: GET
- Аутентификация: Нет
- Описание: Этот маршрут предоставляет информацию о текущих акциях.

3. /menu/categories - Просмотр категорий меню

- HTTP Метод: GET
- Аутентификация: Нет
- Описание: Позволяет клиентам просматривать категории доступных блюд.

Задание 4. Проектирование и тестирование API

1. Изучить примеры запросов из «[Swagger Petstore - OpenAPI 3.0](#)» и примеры из статьи в задании 1.
2. Создать учетную запись на сервисе с поддержкой Open API, например Swagger UI (<https://app.swaggerhub.com/home>) или Postman (<https://www.postman.com>).
3. Создать API, например на SwaggerHub, с нуля или из шаблона, или импортировав существующее API согласно статье <https://support.smartbear.com/swaggerhub/docs/en/manage-apis/creating-a-new-api.html>. Если при создании активирован переключатель для создания Mock-сервера, то он будет автоматически создан. Mock-сервер применяется для симуляции ответов.

Select a Template or create a Blank API

×

Enter a unique name for your API and select a Template. Select 'None' for a Blank API.

[Learn more](#) ↗

ammmates ✕

Owner	DianaMishkova	▼	
	---	None ---	▼
Specification	OpenAPI 3.0.x	▼	
Template	IOT	▼	
Name	AsianRestaurant		
Visibility	Public	▼	

 0/3 public APIs used. [Contact admin to upgrade](#) ↗

Auto Mock API  OFF ☒ ON

4. Определить тело для каждого POST и PUT-запроса, а также статусы HTTP ответов и их смысл в спецификации Open API как на рисунке 5.

POST	<code>/feedback/review</code>	Оставить отзыв	   
POST	<code>/order/submit</code>	Сделать заказ	   
POST	<code>/menu/add_dish</code>	Добавить блюдо	   
PUT	<code>/menu/update_dish/{id}</code>	Изменить блюдо	   
PUT	<code>/order/update_status/{id}</code>	Изменить состояние заказа	   

5. Протестировать разработанную спецификацию. Если Mock-сервер не был активирован в пункте 3 текущего задания, то настройте его согласно документации <https://support.smartbear.com/swaggerhub/docs/en/integrations/api-auto-mocking.html>

POST

/menu/add_dish

Добавить блюдо

Администратор добавляет новое блюдо в меню.

Parameters

No parameters

Request body required

```

{
  "name": "Суп с лапшой и свиной",
  "description": "Сырая свинина отваривается в курином бульоне, вместе с сахаром и рыбным соусом, отчего определить природу будущего супа на вкус становится невозможн
  "price": 20,
  "categoryId": "Суп"
}

```

applicat

Execute

Code	Details
201	Response headers <pre> access-control-allow-credentials: true access-control-allow-headers: X-Requested-With,Content-Type,Accept,Origin access-control-allow-methods: * access-control-allow-origin: * cache-control: no-cache content-encoding: gzip content-length: 20 content-type: application/json;charset=utf-8 date: Thu,14 Nov 2024 22:40:05 GMT etag: W/"14-yIMMzzqGPKico39NpXK600BdB3s" expires: -1 server: nginx x-powered-by: Express </pre> Request duration 757 ms

6. Добавить примеры запросов и ссылку на документацию API в документацию проекта в вики и github pages.

Примеры запросов
1. Просмотр меню Маршрут: <code>/menu</code> Метод: <code>GET</code> Пример запроса: <pre>GET /menu HTTP/1.1 Host: https://virtserver.swaggerhub.com/DianaMishkova/AsianRestaurant/1.0.0</pre> Описание: Возвращает список блюд в меню. 2. Подтверждение заказа Маршрут: <code>/order/submit</code> Метод: <code>POST</code> Пример запроса: <pre>POST /order/submit HTTP/1.1</pre>

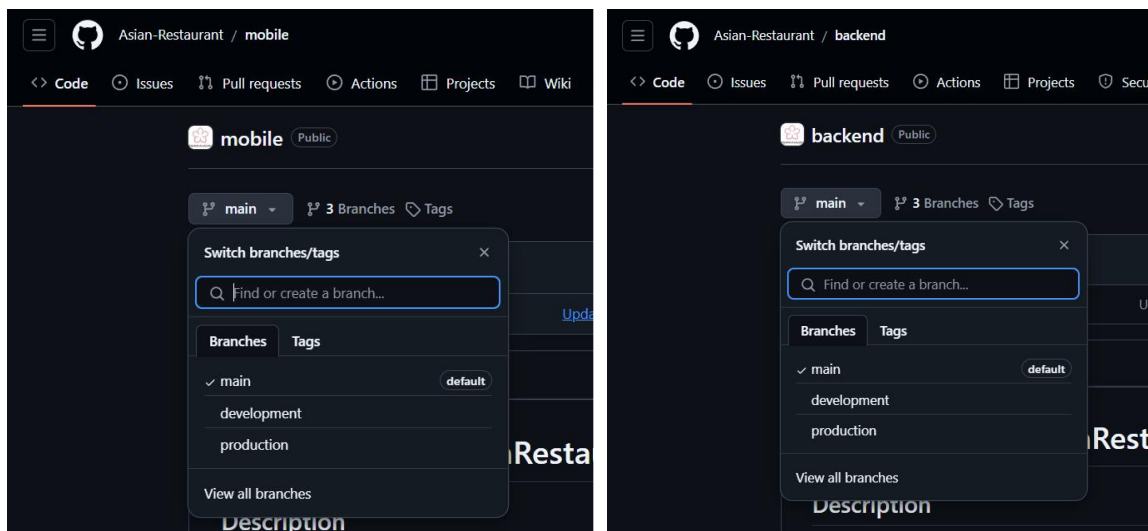
Задание 5. Спроектировать инфраструктуру проекта и развернуть ее

1. Изучить статью и последовательность шагов, необходимых для построения окружения разработки

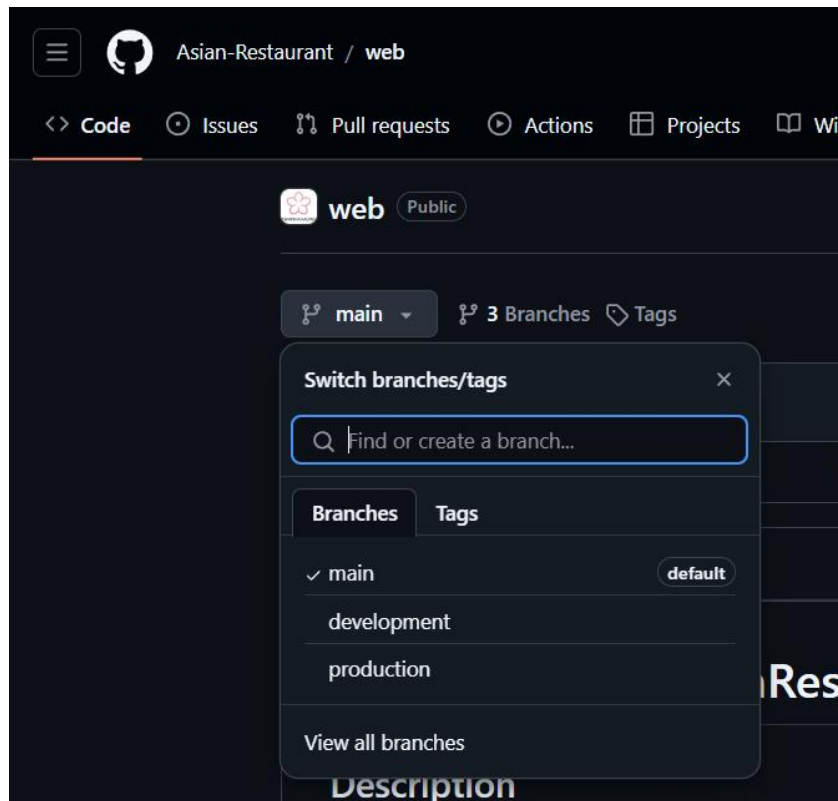
<https://habr.com/ru/company/southbridge/blog/329262/>
<https://selectel.ru/blog/tutorials/develop-canban-board-on-django-drf-alpine-js/>

ИЛИ

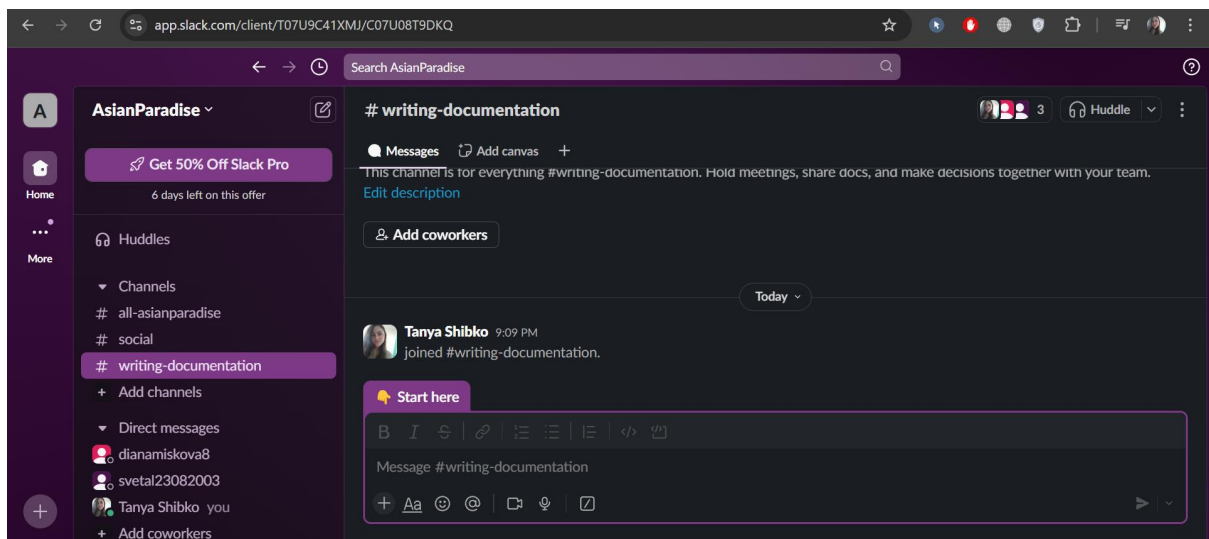
2. Создать структуру проекта в репозиториях для **бекенд**, **веб** и **мобильной версии** проекта на github. Создать ветки для релиз-версии (production) и тестовой версии (development). Согласовать правила работы с ветками. Разработку проекта вести только с



использованием системы контроля версий.



3. Управление проектом вести с использованием Project в github. Для коммуникации использовать Slack или иной мессенджер для командной работы.



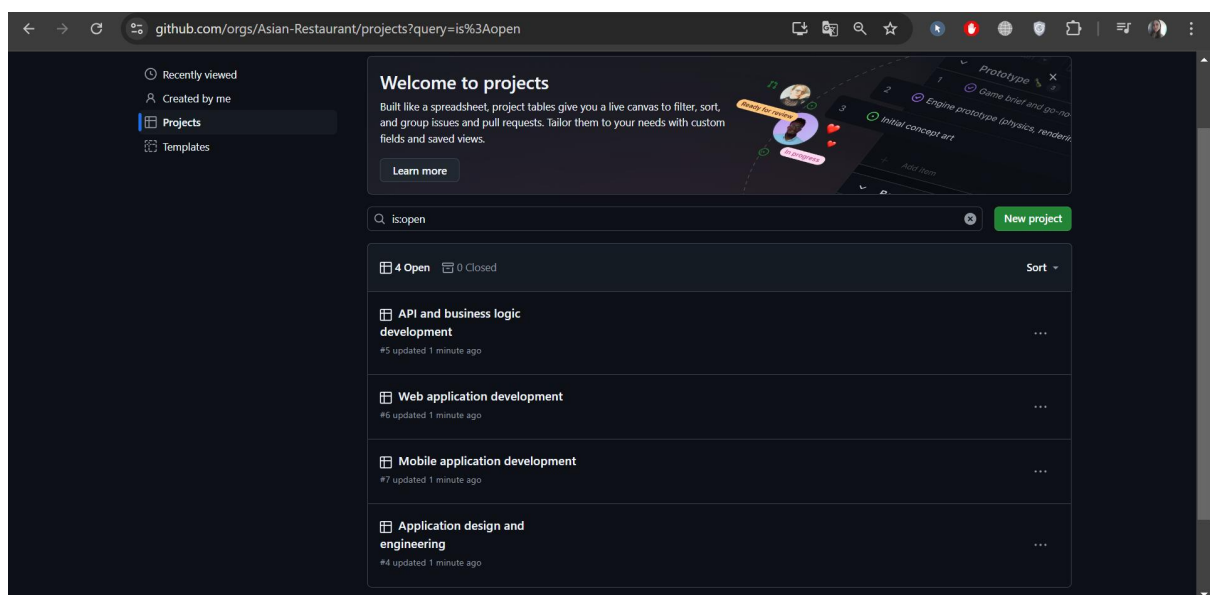
При необходимости создать необходимое количество проектов в основной репозитории, например:

✓ Проект “Дизайн и проектирование приложений” –
общая документация по проекту и отчеты по лабораторным
работам

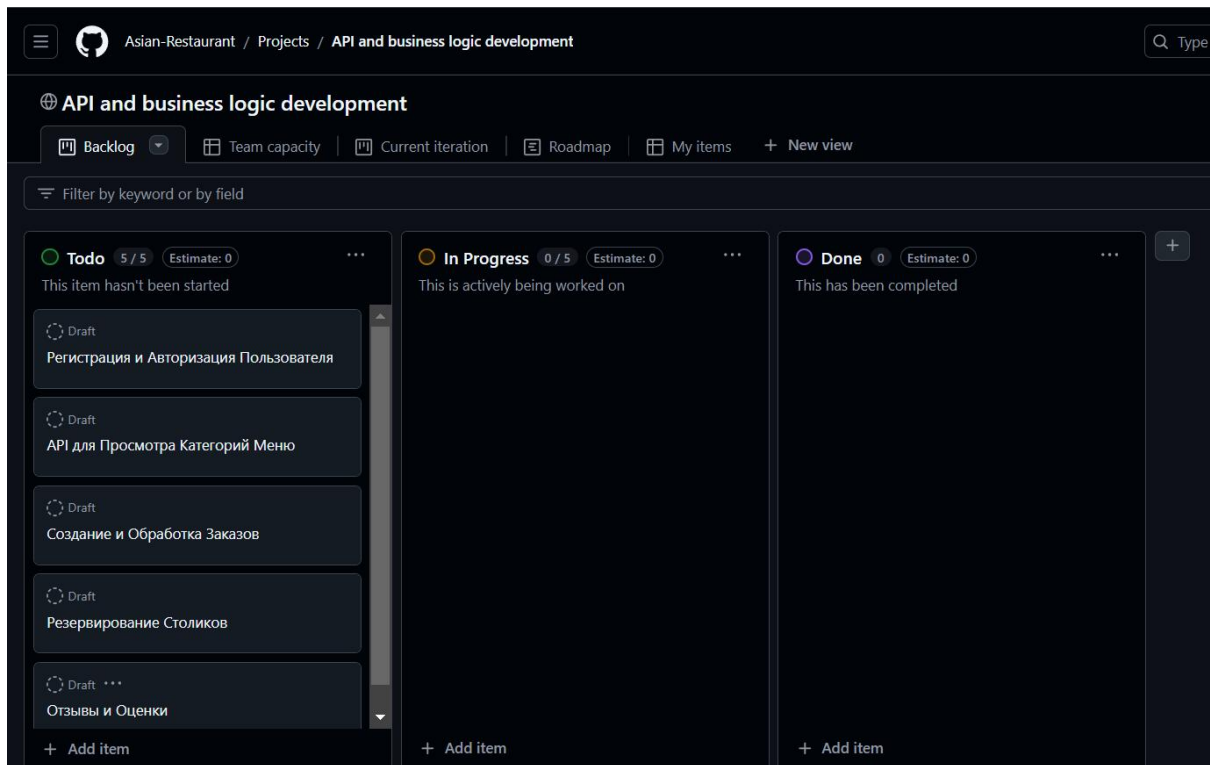
✓ Проект “Разработка API и бизнес-логики”

✓ Проект “Разработка веб-приложения”

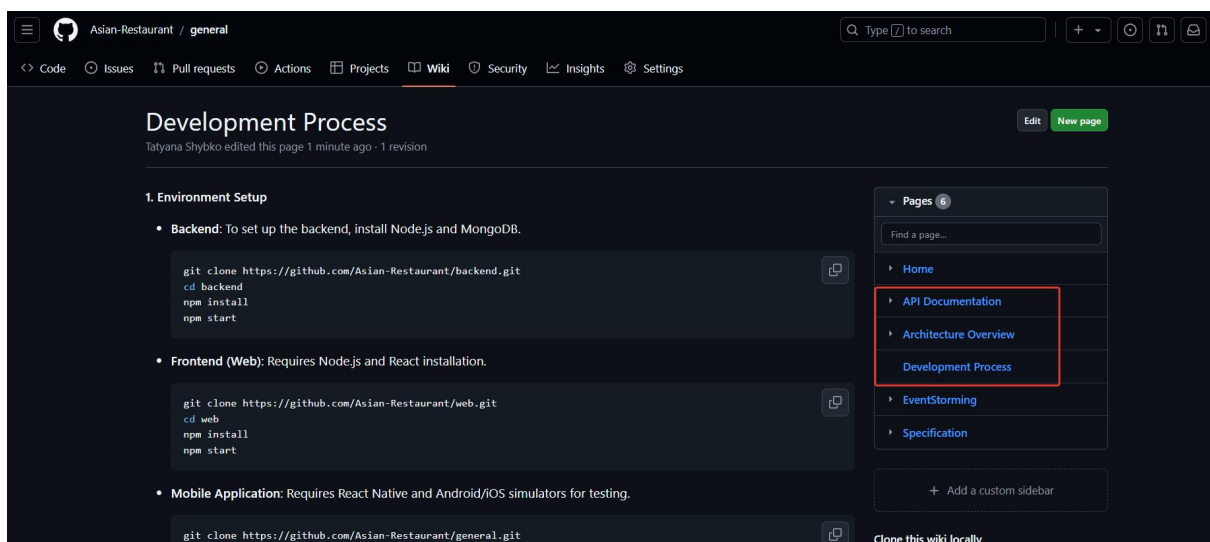
✓ Проект “Разработка мобильного приложения”

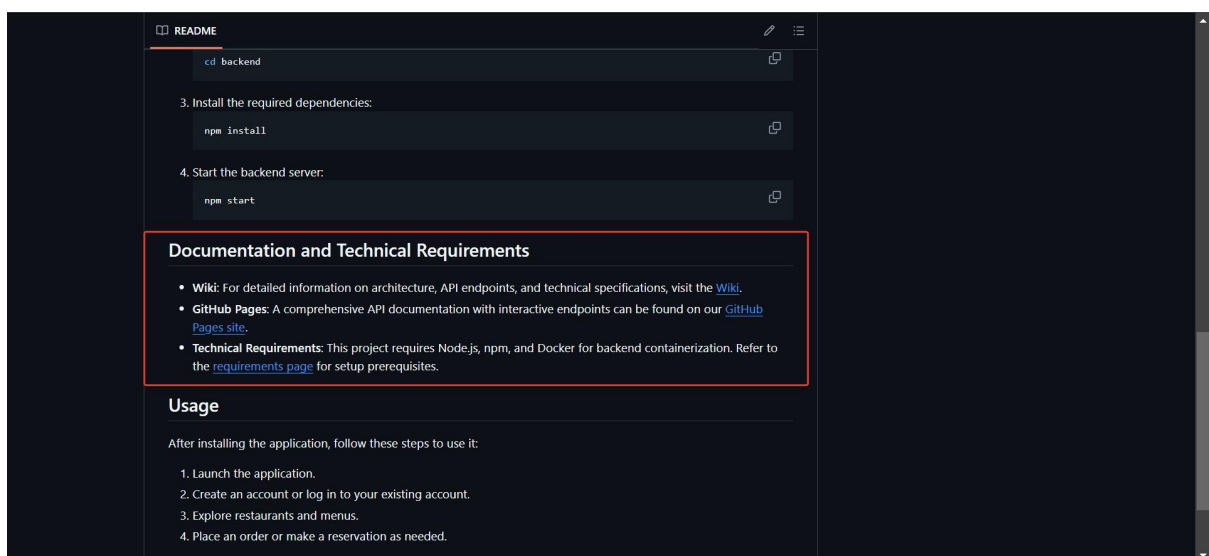
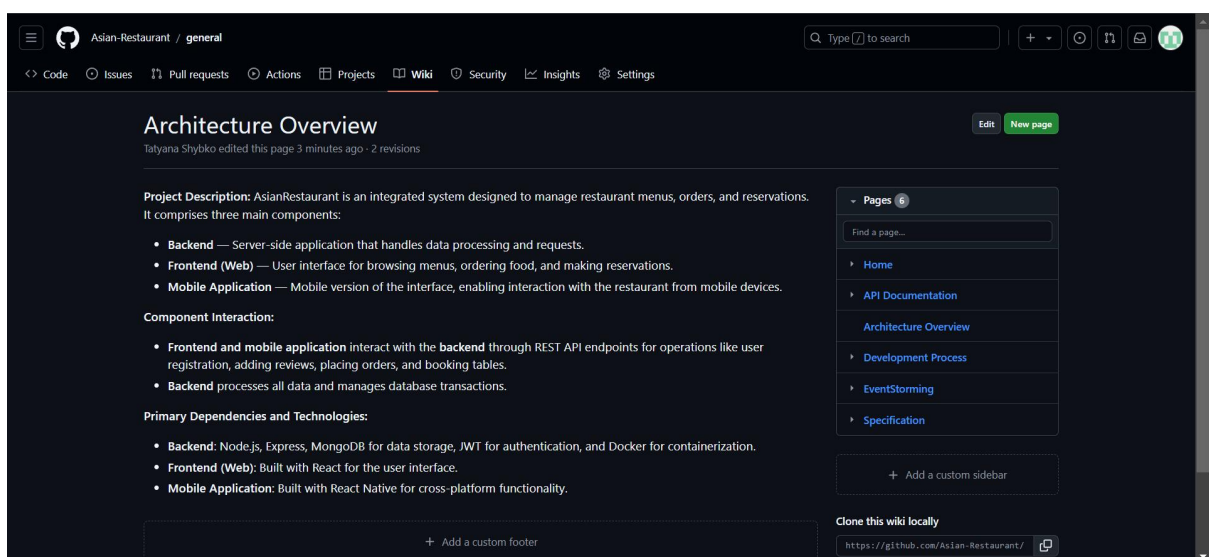
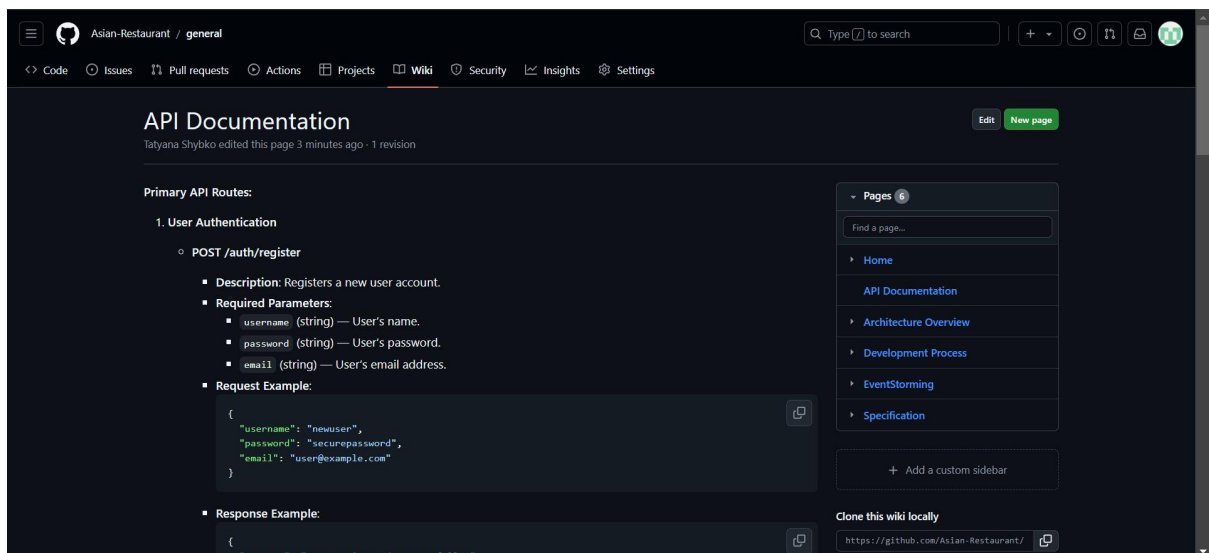


4. На основе технического задания, разработанного в п. 1 данной лабораторной работы, составить план работ по бекенду, включая разработку API, распределив задачи между участниками команды.

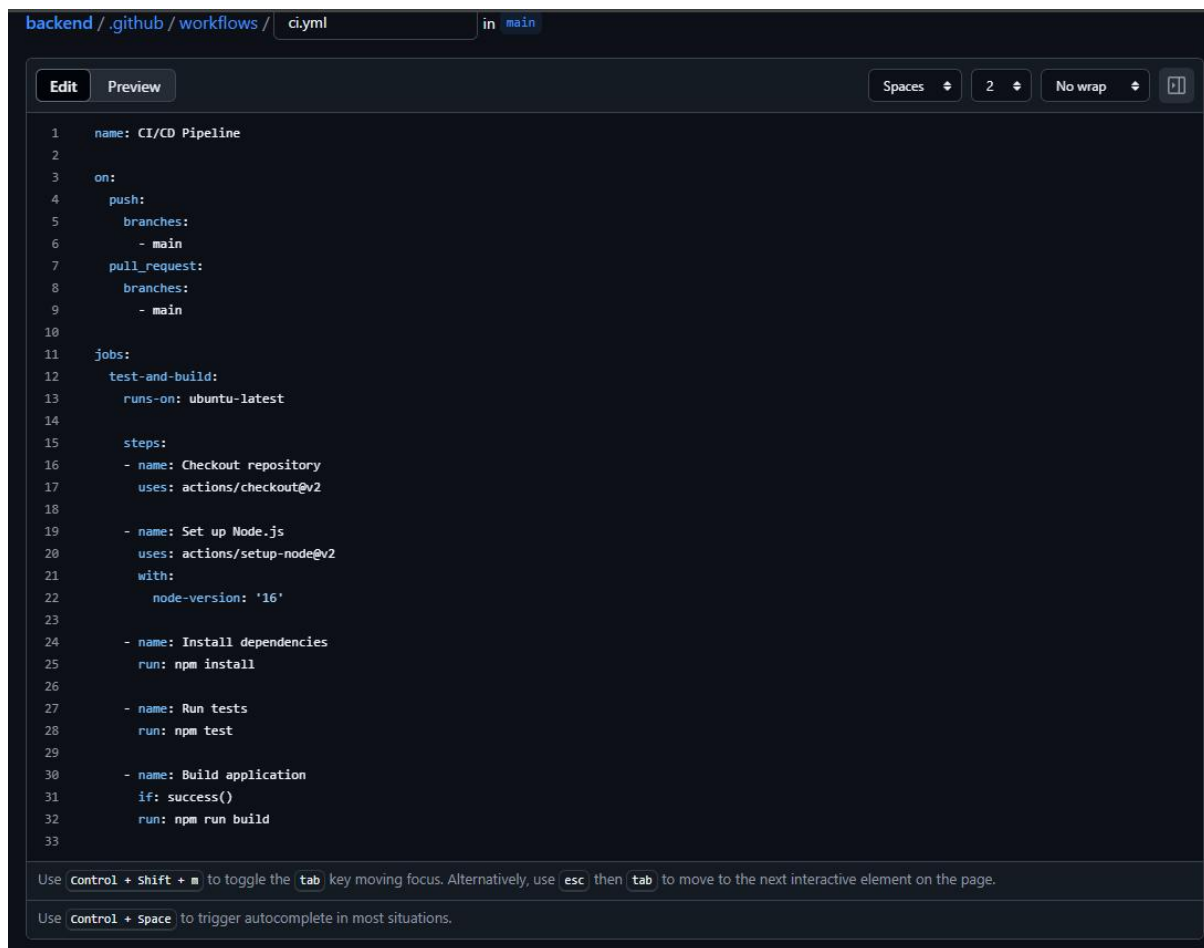


5. План работ добавить в проекты в системе контроля версий в основной репозиторий и документировать проект в Readme, wiki и Github Pages репозитория согласно требованиям, представленным в Документирование проекта.





6. Добавить автоматические тесты, чтобы сервер CI перед сборкой выполнял тесты и формировал сборку только после их прохождения.

The image shows a screenshot of a code editor displaying a GitHub Actions workflow file named `ci.yml`. The editor has a dark theme. At the top, the breadcrumb navigation shows `backend / .github / workflows / ci.yml` and the current branch is `main`. The workflow configuration is as follows:

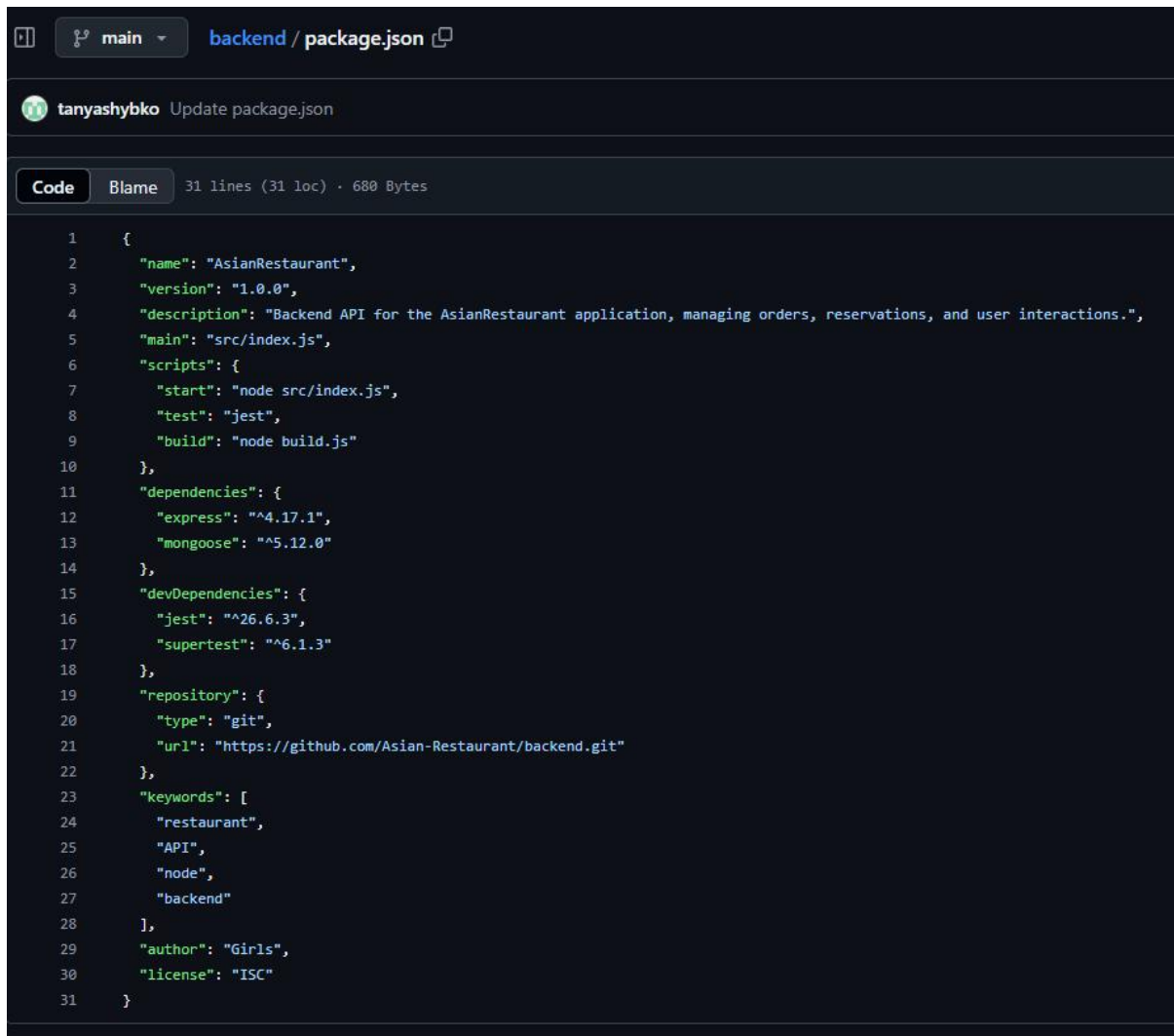
```
1  name: CI/CD Pipeline
2
3  on:
4    push:
5      branches:
6        - main
7    pull_request:
8      branches:
9        - main
10
11  jobs:
12    test-and-build:
13      runs-on: ubuntu-latest
14
15      steps:
16        - name: Checkout repository
17          uses: actions/checkout@v2
18
19        - name: Set up Node.js
20          uses: actions/setup-node@v2
21          with:
22            node-version: '16'
23
24        - name: Install dependencies
25          run: npm install
26
27        - name: Run tests
28          run: npm test
29
30        - name: Build application
31          if: success()
32          run: npm run build
33
```

At the bottom of the editor, there are two lines of instructional text: "Use `Control + Shift + M` to toggle the `tab` key moving focus. Alternatively, use `esc` then `tab` to move to the next interactive element on the page." and "Use `Control + Space` to trigger autocomplete in most situations."

Пояснение этапов конфигурации:

1. Checkout repository: Клонировать текущий репозиторий, чтобы CI мог получить доступ к файлам.
2. Set up Node.js: Устанавливает Node.js версии 16, но вы можете указать другую версию, если нужно.
3. Install dependencies: Устанавливает зависимости, указанные в `package.json`.
4. Run tests: Запускает тесты. Если тесты не проходят, процесс CI завершается с ошибкой, и шаг сборки не будет выполнен.
5. Build application: Если все тесты прошли успешно, запускается сборка приложения.

Добавление файла для выполнения скриптов:



```
1  {
2    "name": "AsianRestaurant",
3    "version": "1.0.0",
4    "description": "Backend API for the AsianRestaurant application, managing orders, reservations, and user interactions.",
5    "main": "src/index.js",
6    "scripts": {
7      "start": "node src/index.js",
8      "test": "jest",
9      "build": "node build.js"
10   },
11   "dependencies": {
12     "express": "^4.17.1",
13     "mongoose": "^5.12.0"
14   },
15   "devDependencies": {
16     "jest": "^26.6.3",
17     "supertest": "^6.1.3"
18   },
19   "repository": {
20     "type": "git",
21     "url": "https://github.com/Asian-Restaurant/backend.git"
22   },
23   "keywords": [
24     "restaurant",
25     "API",
26     "node",
27     "backend"
28   ],
29   "author": "Girls",
30   "license": "ISC"
31 }
```

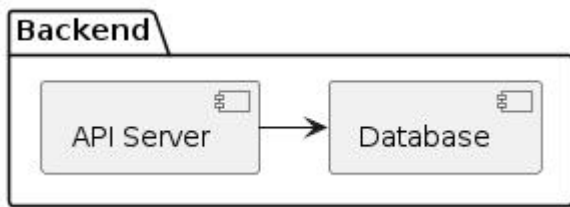
7. Спроектировать инфраструктуру бекенд и веб-приложения на основе системы контейнеризации Docker инфраструктуру, используя PlanUML. Если диаграммы спроектирована в лабораторной работе 3, уточнить ее, при необходимости.

1. Настроим контейнер для backend

- Цель: Этот контейнер будет содержать API сервер, обрабатывать запросы от клиента, управлять данными и взаимодействовать с базой данных.

Обозначение в PlanUML:

```
package Backend {
  [API Server] -right-> [Database]
}
```



2. Настроим контейнер базы данных

- Цель: Разделить базу данных в отдельный контейнер, чтобы гарантировать безопасность и лёгкость управления.

Обозначение в PlanUML:

```

package Database {
  [Database: PostgreSQL/MySQL]
}
  
```



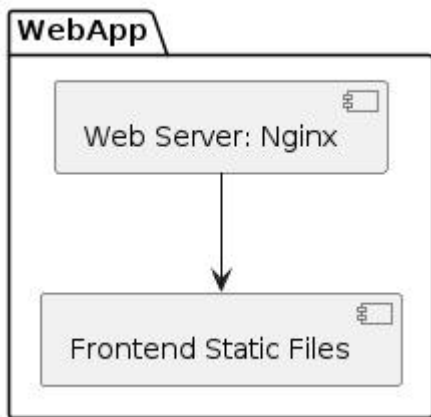
3. Настроим контейнер для web-приложения

- Цель: Веб-сервер (например, Nginx) будет обслуживать статические файлы и обеспечивать поддержку клиентских запросов.

Обозначение в PlanUML:

```

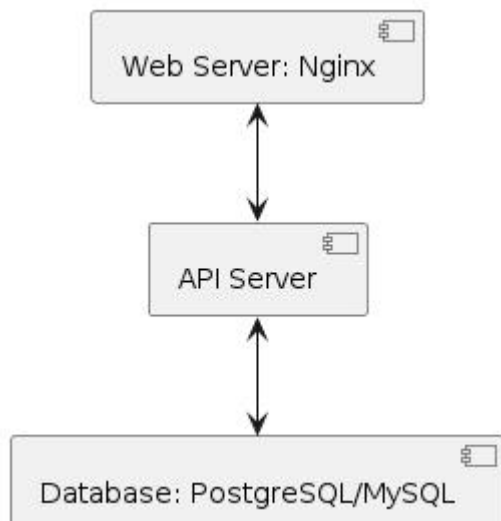
package WebApp {
  [Web Server: Nginx] --> [Frontend Static Files]
}
  
```



4. Настроим сеть Docker

- Цель: Связь между backend, web-приложением и базой данных через сеть Docker для внутренних коммуникаций между контейнерами.

Обозначение В PlanUML:
[API Server] <--> [Database: PostgreSQL/MySQL]
[Web Server: Nginx] <--> [API Server]



5. Настроим CI/CD сервер

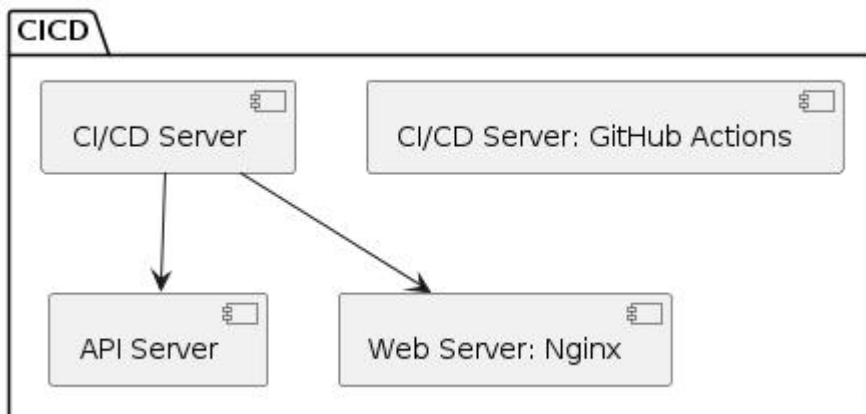
- Цель: Сервер CI/CD (например, GitHub Actions или Jenkins) будет автоматически тестировать и разворачивать контейнеры.

Обозначение В PlanUML:
package CICD {

```

[CI/CD Server: GitHub Actions]
[CI/CD Server] -down-> [API Server]
[CI/CD Server] -down-> [Web Server: Nginx]
}

```



6. Добавляем логирование и мониторинг

- Цель: ELK Stack или другие инструменты для логирования и мониторинга обеспечат наблюдение за работой контейнеров.

Обозначение

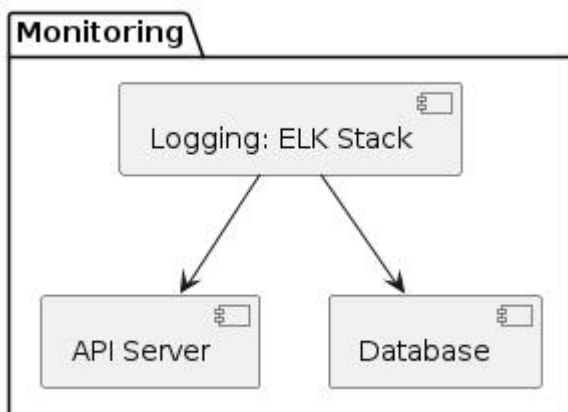
в

PlanUML:

```

package Monitoring {
    [Logging: ELK Stack] --> [API Server]
    [Logging: ELK Stack] --> [Database]
}

```



7. Оркестрация контейнеров

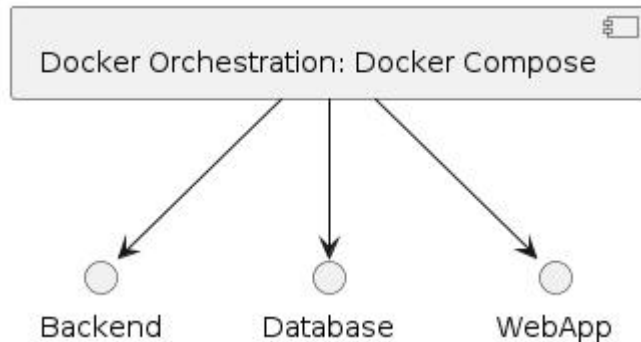
- Цель: Использовать Docker Compose или Docker Swarm для управления и развертывания нескольких контейнеров.

Обозначение в PlanUML:

[Docker Orchestration: Docker Compose] --> Backend

[Docker Orchestration: Docker Compose] --> Database

[Docker Orchestration: Docker Compose] --> WebApp



Итоговая диаграмма PlanUML

Объединив всё вышеуказанное, можно представить итоговую инфраструктурную диаграмму следующим образом:

@startuml

package Backend {

[API Server] -right-> [Database]

}

package Database {

[Database: PostgreSQL/MySQL]

}

package WebApp {

[Web Server: Nginx] --> [Frontend Static Files]

}

package CICD {

[CI/CD Server: GitHub Actions]

[CI/CD Server] -down-> [API Server]

[CI/CD Server] -down-> [Web Server: Nginx]

}

package Monitoring {

[Logging: ELK Stack] --> [API Server]

[Logging: ELK Stack] --> [Database]

}

[API Server] <--> [Database: PostgreSQL/MySQL]

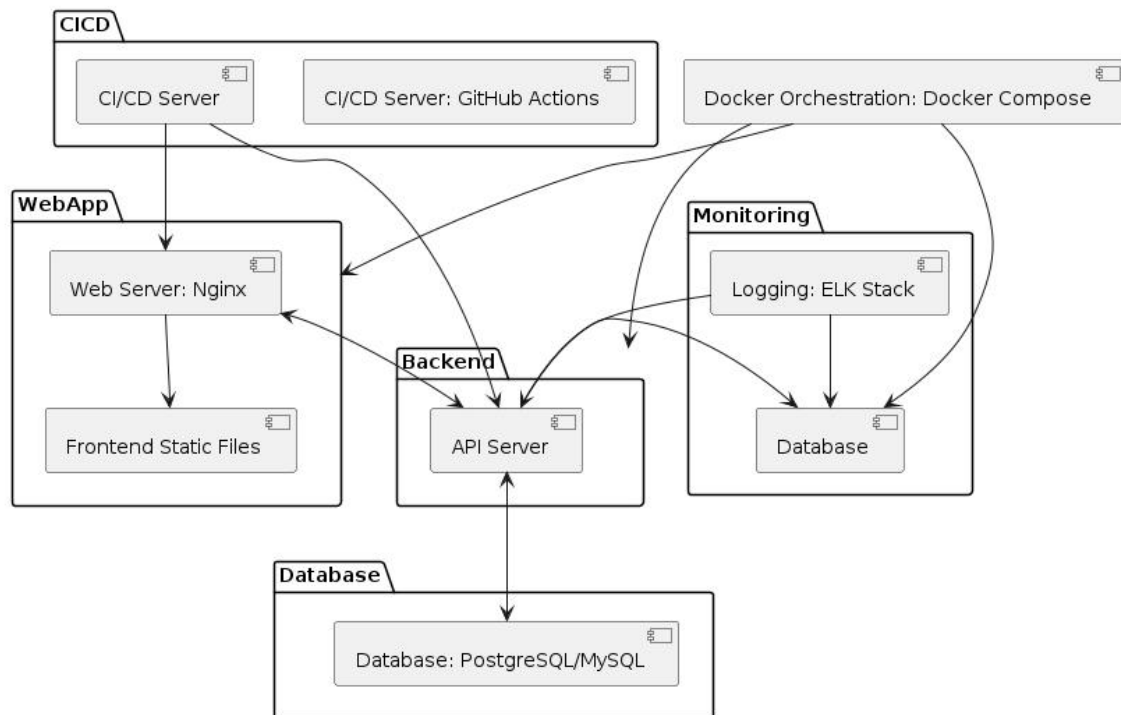
[Web Server: Nginx] <--> [API Server]

[Docker Orchestration: Docker Compose] --> Backend

[Docker Orchestration: Docker Compose] --> Database

[Docker Orchestration: Docker Compose] --> WebApp

@enduml



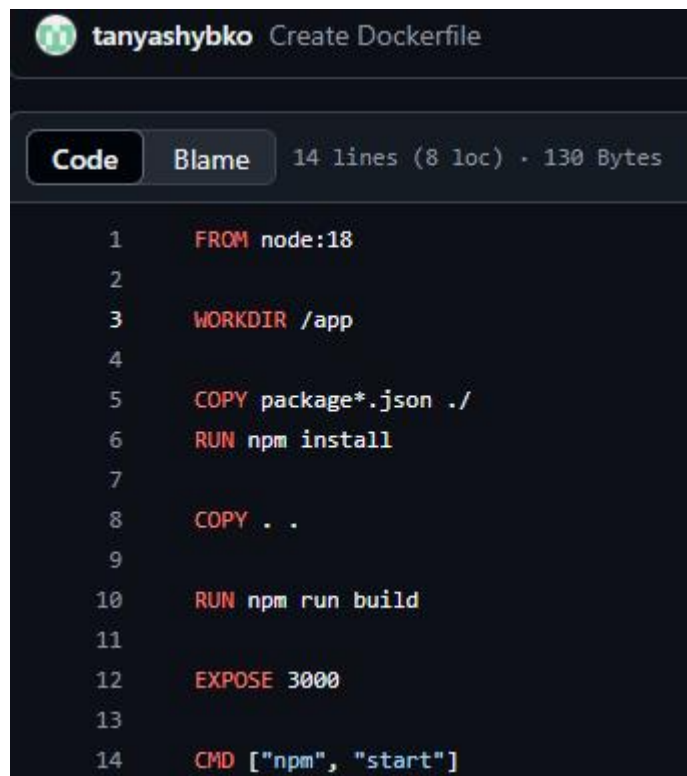
8. Развернуть инфраструктуру для бекенд, веб-приложения на основе контейнеров, конфигурацию которых описать в конфигурационных файлах Dockerfile и для запуска использовать Docker Compose.

Dockerfile для бекенда

```

Code Blame 12 lines (7 loc) · 111 Bytes
1 FROM node:18
2
3 WORKDIR /app
4
5 COPY package*.json ./
6 RUN npm install
7
8 COPY . .|
9
10 EXPOSE 5000
11
12 CMD ["npm", "start"]
  
```

Dockerfile для web



The image shows a GitHub repository interface for a file named 'Create Dockerfile' by user 'tanyashybko'. The file is 14 lines long, 8 lines of code, and 130 bytes. The code is a Dockerfile with the following instructions:

```
1 FROM node:18
2
3 WORKDIR /app
4
5 COPY package*.json ./
6 RUN npm install
7
8 COPY . .
9
10 RUN npm run build
11
12 EXPOSE 3000
13
14 CMD ["npm", "start"]
```

docker-compose.yml


```
Code Blame 27 lines (21 loc) · 524 Bytes

1  name: Docker Compose CI
2
3  on:
4    push:
5      branches:
6        - main
7    pull_request:
8
9  jobs:
10   build:
11     runs-on: ubuntu-latest
12
13     steps:
14       - name: Check out the repository
15         uses: actions/checkout@v2
16
17       - name: Set up Docker Buildx
18         uses: docker/setup-buildx-action@v1
19
20       - name: Log in to Docker Hub
21         uses: docker/login-action@v1
22         with:
23           username: ${ secrets.DOCKER_USERNAME }
24           password: ${ secrets.DOCKER_PASSWORD }
25
26       - name: Build and push
27         run: docker-compose up --build
```

9. Добавить простое приложение типа «Hello World» для иллюстрации работоспособности каждого настроенного окружения.

Для проверки всего я добавила новый репозиторий, чтоб другие не засоряют.

hello-world-app / index.js



tanyashybko Create index.js

Code

Blame

11 lines (9 loc) · 255 Bytes

```
1  const express = require('express');
2  const app = express();
3
4  app.get('/', (req, res) => {
5    res.send('Hello, World!');
6  });
7
8  const PORT = process.env.PORT || 3000;
9  app.listen(PORT, () => {
10    console.log(`Server is running on http://localhost:${PORT}`);
11  });
```

hello-world-app / package.json



tanyashybko Create package.json

Code

Blame

11 lines (11 loc) · 175 Bytes

```
1  {
2    "name": "hello-world-app",
3    "version": "1.0.0",
4    "main": "index.js",
5    "scripts": {
6      "start": "node index.js"
7    },
8    "dependencies": {
9      "express": "^4.17.1"
10   }
11 }
```

hello-world-app / Dockerfile



tanyashybko Create Dockerfile

Code

Blame

18 lines (13 loc) · 556 Bytes

```
1  # Используем официальный образ Node.js как базовый
2  FROM node:14
3
4  # Устанавливаем рабочую директорию в контейнере
5  WORKDIR /app
6
7  # Копируем package.json и устанавливаем зависимости
8  COPY package.json ./
9  RUN npm install
10
11 # Копируем весь код приложения
12 COPY . .
13
14 # Указываем, что контейнер будет слушать на порту 3000
15 EXPOSE 3000
16
17 # Запускаем сервер
18 CMD ["npm", "start"]
```

hello-world-app / .dockerignore



tanyashybko Create .dockerignore

Code

Blame

2 lines (2 loc) · 27 Bytes

```
1  node_modules
2  npm-debug.log
```

hello-world-app / .github / workflows / hello-world.yml 



tanyashybko Update hello-world.yml ✓

Code

Blame

32 lines (25 loc) · 668 Bytes

```
1  name: CI/CD Pipeline
2
3  on:
4    push:
5      branches:
6        - main
7
8  jobs:
9    build:
10     runs-on: ubuntu-latest
11
12     steps:
13       - name: Checkout code
14         uses: actions/checkout@v2
15
16       - name: Set up Docker
17         uses: docker/setup-buildx-action@v2
18
19       - name: Log in to Docker Hub
20         uses: docker/login-action@v2
21         with:
22           username: ${ secrets.DOCKER_USERNAME }
23           password: ${ secrets.DOCKER_PASSWORD }
24
25       - name: Install Docker Compose
26         run: |
27           sudo apt-get update
28           sudo apt-get install -y docker-compose
29
30       - name: Build and run containers with Docker Compose
31         run: |
32           docker-compose -f docker-compose.yml up -d
```

hello-world-app / .github / workflows / docker.yml

tanyashybko Update docker.yml

Code Blame 39 lines (38 loc) · 886 Bytes

```
1  name: Docker Build and Push
2
3  on:
4    push:
5      branches:
6        - main
7
8  jobs:
9    build:
10     runs-on: ubuntu-latest
11
12     steps:
13     - name: Checkout code
14       uses: actions/checkout@v2
15
16     - name: Set up Docker Buildx
17       uses: docker/setup-buildx-action@v2
18
19     - name: Log in to Docker Hub
20       uses: docker/login-action@v2
21       with:
22         username: ${ secrets.DOCKER_USERNAME }}
23         password: ${ secrets.DOCKER_PASSWORD }}
24
25     - name: Install Docker Compose
26       run: |
27         sudo apt-get update
28         sudo apt-get install -y docker-compose
29
30     - name: Build Docker image
31       run: docker build -t hello-world-app .
32
33     - name: Run Docker Compose
34       run: |
35         docker-compose up -d
36
37     - name: Verify the application is running
38       run: |
39         curl http://localhost:3000
```

Проверяем работоспособность

 Update docker.yml CI/CD Pipeline #10: Commit 0bd7d68 pushed by tanyashybko	main	8 minutes ago 54s	...
 Update hello-world.yml CI/CD Pipeline #8: Commit e47ede1 pushed by tanyashybko	main	18 minutes ago 54s	...

10. Для автоматизации сборки проекта настроить сервис непрерывной интеграции (сервис Github Actions или Gitlab CI/CD). Обеспечить сборку с docker-контейнерами.

11. Доработать процессы CI/CD и встроить генерацию необходимых файлов для Swagger одним из этапов сборки.

12. Изучить статьи <https://www.codeflow.site/ru/article/sonar-qube> и https://habr.com/ru/companies/sportmaster_lab/articles/746320/. Настроить сервис для проверки качества кода SonarQube для локальных версий проектов бекенд и веб-приложение и облачную версию для включения в качестве этапа в CI/CD.

13. Обеспечить развертывание проекта (бекенд) из контейнеров Docker после прохождения тестов и проверки качества кода на бесплатных платформах, например <https://render.com/> или <https://vercel.com/>. Веб-приложение может быть локальным или развернуто на другой учетной записи любого из сервисов в бесплатном тарифе.

14. Создание отчета, описывающего работы по всем пунктам данного задания и вклад каждого участника проекта, включая информацию по коммитам из git репозитория и распределение работ в рамках проекта.

15. Для публикации на внешнем хостинге можно использовать бесплатные облачные платформы из пункта 13. Для развертывания окружения разработки локально использовать контейнеры на примере Docker или LXC/LXD. Продемонстрировать подключение к окружению разработки и запуск приложения на облачном сервисе, включая API, тесты, проверку качества кода и процесс CI/CD, настроенный в Github (по желанию Gitlab).